PROJECT 2 | APPLIED AI + NETWORK TROUBLESHOOTING

NetSage AI

Build an AI troubleshooting helper with human review

In one sentence: Create an AI-assisted troubleshooter for Packet Tracer lab problems that reads symptoms and show-command output, suggests likely causes and next steps, and always requires a human to review before accepting the fix.

Problem Statement

Junior network engineers often know individual commands but struggle to connect a symptom to the real root cause. When a PC gets an IP address but cannot reach a server, is the problem VLAN, routing, DHCP, DNS, ACL, or NAT?

Your team must build a troubleshooting assistant for Cisco-style lab networks. The assistant should use symptoms, Packet Tracer notes, and show-command outputs to recommend a likely fault, the OSI layer, the next command to run, and an evidence-backed fix. A human reviewer must approve or correct every diagnosis.

MAIN COURSE	DOMAIN	SAFETY RULE	TEAM SIZE
Modern AI	**Networking labs**	**Human review**	**2-3 students**

What You Must Build

Component	Requirement
Case dataset	At least 30 troubleshooting cases from Packet Tracer or lab scenarios
Evidence per case	Symptom, topology note, show outputs, expected fault, OSI layer, concept tag
AI prompt library	Structured prompts that return root cause, confidence, evidence, next command, fix
Rule checker	Python script with deterministic checks for common config mistakes
Dashboard	Simple summary of issue types, severity, and AI vs human agreement
Responsible AI log	At least 5 cases where the AI answer was corrected by a human

Step-by-Step Workflow

1. **Collect real lab cases:** Create at least 30 cases covering VLAN, gateway, DHCP, DNS, routing, ACL, NAT, and wireless issues.
2. **Write structured prompts:** Design prompts that force JSON output with fields like root_cause, confidence, evidence, next_command, and fix_steps. Include 2 or 3 worked examples.
3. **Build the rule checker:** Use Python to check duplicate IPs, wrong masks, gateway mismatch, interface down, missing VLAN, and missing routes before or after AI diagnosis.
4. **Run AI diagnosis:** Feed each case to the AI assistant. Save the response and compare it with the known correct answer.
5. **Add human review:** Mark each case as Accepted, Edited, or Rejected. Log cases where AI was wrong and explain why.
6. **Build the dashboard and demo:** Show counts by issue type and a demo of one broken lab being diagnosed, reviewed, fixed, and verified.

Deliverables

Item	What to submit
cases.csv	All cases with symptom, show outputs, expected fault, OSI layer, concept, severity
Prompt files	diagnose_prompt.md and any helper prompt templates

Python checker	Rule-based validation script with sample output
Dashboard	Spreadsheet or simple chart showing themes and AI agreement rate
Responsible AI log	Notes on at least 5 corrected AI responses
Demo video	5 to 10 minutes showing broken case, AI output, human review, fix, verification

How Your Work Will Be Checked

Check	Pass condition
Case coverage	At least 30 cases across multiple network fault types
Evidence use	AI responses quote or reference actual show-command evidence
Human oversight	Reviewer log shows accepted, edited, and rejected diagnoses
Deterministic checks	Python checker catches basic config errors correctly
Responsible AI	Team documents at least 5 cases where AI needed correction

Example diagnosis

Symptom	Expected response
PC gets IP but cannot reach server in VLAN 30; gateway ping works	Likely inter-VLAN routing or ACL issue at Layer 3/4. Next commands: show ip route, show access-lists, show interfaces trunk. Confidence: medium until route/ACL evidence is shown.
Guest Wi-Fi can reach internal server	Likely guest isolation failure. Security issue. Next: inspect VLAN mapping and ACL rules.